American Sign Language Image Classification

Matthew Cruz, Marco Parete, Scott Lowder, and Ivan Chiu
CSCI3345 Machine Learning
Prof. Sergio Alvarez
December 8, 2025

Abstract

American Sign Language (ASL) is a primary form of communication for many people who are deaf, have deaf family members or friends, or are hard-of-hearing. This project aims to create a machine learning model that can successfully classify images of ASL hand signs. Many different types of models were used, tested, and compared to find the most accurate, using the cross-validated out-of-sample error rates of each class, including Logistic Regression, Random Forest, and Convolution Neural Network models. Two different datasets were also experimented with in order to compare the model's ability to classify images outside the overall dataset, with one dataset having a low variance, and one being rich and diverse in order to more accurately reflect real-world use cases. The images were also randomly mirrored, discolored, or augmented in other ways to further increase the data variance and ability for the model to generalize. Techniques were also used as an attempt at feature extraction, which is notably difficult to infer with image recognition tasks, as there isn't concrete quantitative data, but instead only an image as the input. The result of the project was a highly accurate model capable of classifying real-world images of ASL signs.

Introduction

The development of automated communication tools is a critical task for improving accessibility and integration within the Deaf and hard-of-hearing communities, making machine recognition of sign language a topic of high societal importance and active research (Cheok et al.). This project addresses the fundamental computer vision problem of multi-class classification applied to American Sign Language (ASL), specifically focusing on the translation of static hand signs into English letters. Our primary goal is to train a robust machine learning model capable of accurately classifying 29 distinct classes (A-Z plus 'space', 'delete', and 'nothing'), with an initial performance target set to achieve an accuracy exceeding 90% measured through cross-validation, aligning with benchmarks established in the field (Koller et al.). Key challenges in this task include managing the high dimensionality of raw image data, ensuring the model's generalization to new users with varying hand shapes and lighting conditions, and minimizing confusion between visually similar signs. To address these, we initially established performance

baselines using traditional methods like Logistic Regression and Random Forest, but quickly shifted focus to more advanced image processing techniques. We prioritized a Convolutional Neural Network (CNN) architecture for its proven efficacy in extracting hierarchical features from raw pixels. Furthermore, to gain interpretability and address the issue of similar-sign confusion, we used Grad-CAM (Gradient-weighted Class Activation Mapping) to visualize the features driving the model's decisions (Garg et al.). The following sections detail our methodology, present the comparative results across various model types and datasets, and conclude with the final model that successfully meets the requirements for real-world ASL image classification.

Methods

1st Dataset

The first dataset we used contained 87,000 RGB images at 200×200 resolution, all from the same individual. There were 29 classifications (A–Z plus “Delete,” “Space,” and “Nothing”). From looking through the data, it was clear that the person moved their hand around intentionally to introduce variation in positioning and lighting.

Logistic Regression

Logistic regression was used as a baseline model for classifying American Sign Language (ASL) alphabet images. This method identifies patterns in the data and predicts the class label. For the multi-class problem (29 classes), this was done by using the One-vs-Rest (OvR) strategy. OvR breaks the problem down into 29 separate binary classifiers, where each classifier is independently trained to distinguish one specific class (the “One”) from all other classes (the “Rest”); the final prediction is determined by the classifier that yields the highest probability score for the input. It uses the sigmoid function to convert weighted input scores into probabilities, providing confidence estimates for each prediction (Bishop, C. M.). The model was trained on raw pixel values from resized images (128x128x3, resulting in 49,152 features per image) to ensure computational efficiency. Data preprocessing involved loading the ASL alphabet dataset from local storage, applying transformations including resizing and flattening, and standardizing features using StandardScaler. A stratified k-fold cross-validation (k=4) was performed with a sample limit of 1,000 images per class, meaning the cross-validation folds were created such that each fold contained the same proportion of samples of each class as the full dataset. The method used LogisticRegression from scikit-learn with parameters such as max_iter=500, C=0.5 for regularization. The final model was trained on an 80-20 stratified split of the full dataset, evaluated on test data, and saved along with the scaler for future inference. The feature importance of each pixel was shown by converting the model's coefficients into an image format and averaging the values for the red, green, and blue color channels.

Random Forest

Random Forest served as another baseline model for ASL alphabet classification, building on decision trees to improve reliability and accuracy over single models. It constructs multiple decision trees, each trained on subsets of the data, and makes predictions through majority voting, inherently handling multi-class problems without explicit "one-vs-rest" but effectively capturing complex patterns by feature splits (Breiman, L.). Unlike logistic regression, it does not use a sigmoid function but provides probability estimates by vote proportions across trees, showing prediction confidence. The model was trained on raw pixel values from resized images (128x128x3, yielding 49,152 features per image). Preprocessing was the same logistic regression, including dataset loading, transformations (resizing and flattening), and feature standardization. Stratified k-fold cross-validation (k=4) was conducted with again 1,000 images per class, using RandomForestClassifier from scikit-learn with hyperparameters like n_estimators=20, max_depth=10, min_samples_split=5, min_samples_leaf=2 and max_features='sqrt'. The final model used 50 estimators on an 80-20 stratified split, with out-of-bag scoring enabled, meaning predictions were made for each data point using only the trees that were not trained on that specific point, providing a better estimate of generalization error. Feature importances were analyzed and visualized by reshaping into RGB channels and plotting distributions, top features, and averages.

1st CNN

Before training, we increased the diversity of the dataset through preprocessing. For each image, we allow for there to be a 30% chance of a horizontal flip, random rotation of ±10 degrees, and brightness and contrast are allowed to fluctuate by 20%. This choice is made to expose the model to altered versions of each sign so that when it is used in a real-world application, someone using the model, whether left-handed, slightly rotated, or under different lighting and skin tone, can still be classified correctly. All images, regardless of training or classifying, are resized to 128×128 pixels and normalized using ImageNet statistics (See Figure 1 for an example of how an image is changed before training and after resizing and normalization). This preprocessing exposes the model to slightly altered versions of each sign with the goal of improving generalization to bring out-of-sample error down.

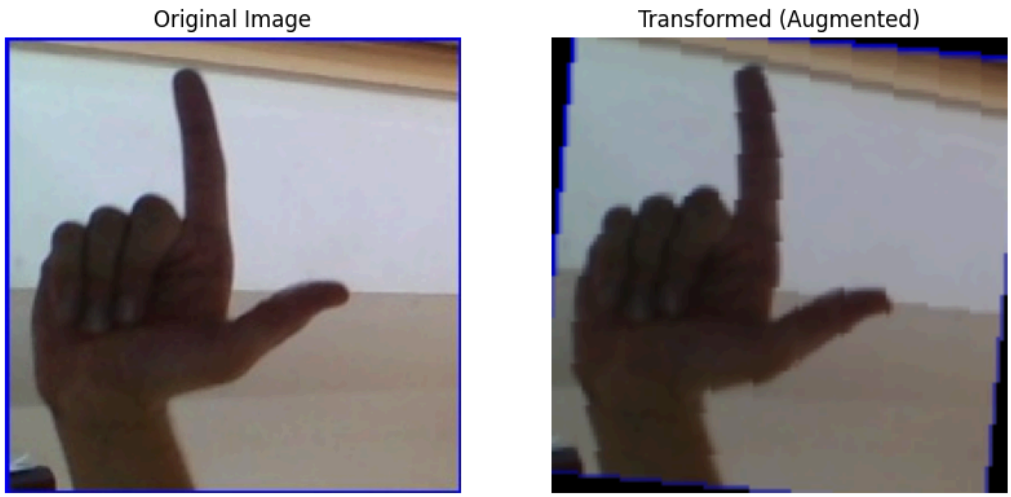


Figure 1. Hand signal after pre-processing

The model's architecture uses channel expansion from 32 to 256 channels, doubling at each stage. After each filter, we run batch normalization and a ReLU activation function, and we use maxpool twice in this process to reduce the spatial size (see Figure 2 to see how the model's spatial size changes). All of this is done to get rid of unimportant features through ReLU while keeping the dominant ones and preventing them from getting too large. This way, the filters can go from identifying edges of fingers to combining fingers to identify hand position and the letter that is being signed. We selected 256 final channels for several reasons. First, it is more efficient for the model than a 512-feature vector, saving on both time and space for what was deemed to be a negligible return. We also noticed that the 128-feature vector would overfit since its capacity was not as large, leading to an increased generalization gap. The depth of 256 allows the network to capture detailed hand-shape patterns while remaining fast enough for real-time ASL interpretation. After feature extraction, the 256-dimensional vector is passed through two linear layers: the first with 128 neurons, and the final output layer with 29 neurons (one per class). We use cross-entropy loss to calculate loss, and then we use the AdamW optimizer to update the weights through backpropagation. The AdamW optimizer is used over the Adam optimizer since it uses regularization to prevent any one weight from having too large an impact. The network does not apply Softmax at the end because PyTorch's CrossEntropyLoss applies Softmax internally during training. In our results, we classify our own photos using a softmax function to give a probability of each letter.

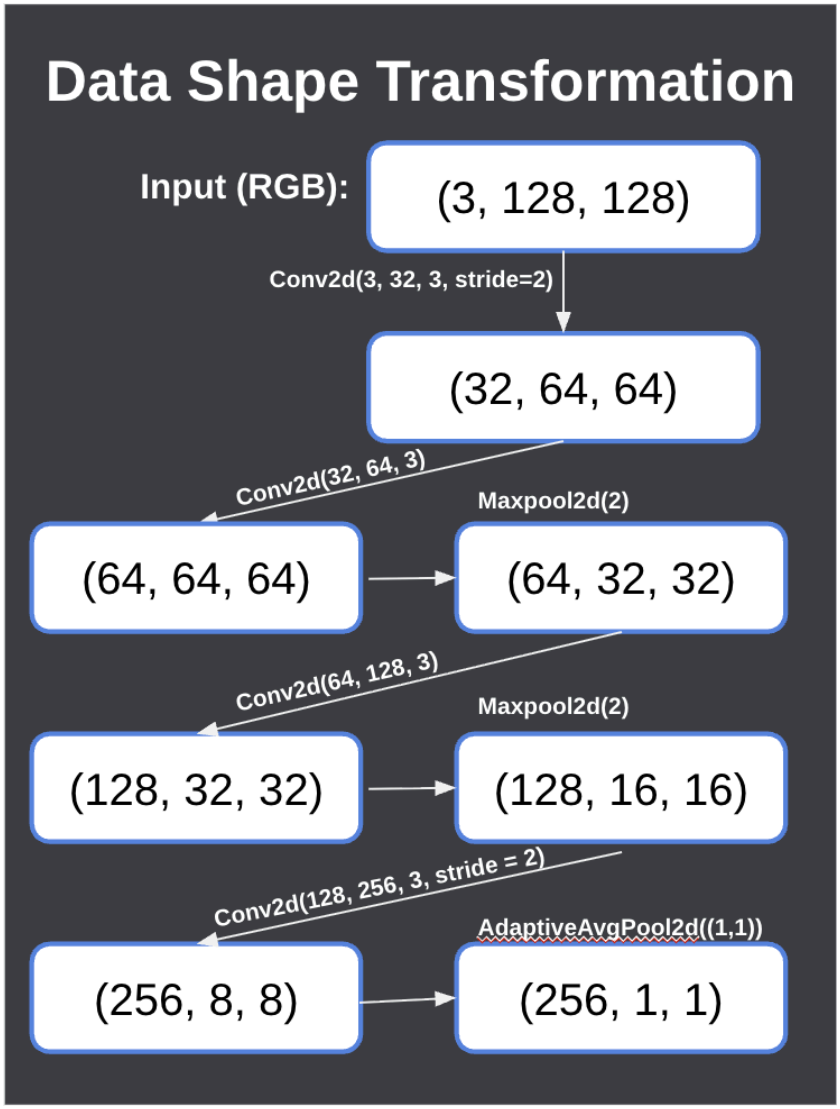


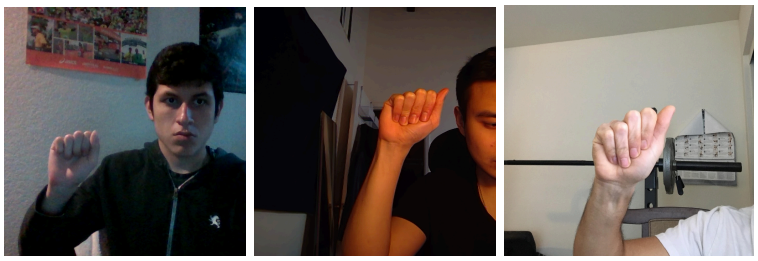
Figure 2. Data shape transformation

2nd Dataset

While we were evaluating the convolutional neural network trained on the first dataset by inputting our own photos of ASL letters, we noticed that very rarely, and less than the model's cross-validation determined out-of-sample error rate, the classification was correct. After looking at the dataset the model was trained on, it became obvious that the almost 90,000 images had very little variance. Upon closer inspection, all the images are the same person's hand, in the same room, with the same camera angle, only slightly varying the distance from the camera, which also affects the lighting on the hand. Due to this oversight in the data selection, the model

had trouble classifying any image that didn't almost exactly mimic the test data, which is not ideal for real-world image classification.

Old dataset



New dataset

Figure 3. Comparison between both datasets

With this in mind, the group thought it would be best to re-evaluate the main dataset used for training, with the goal in mind to have a high-accuracy model that can better classify real-world images from outside the dataset. To accomplish this, the model would need to train on a multitude of diverse images, all with varying skin tones, hand sizes, lighting, angles, backgrounds, and more. The new dataset, also found on Kaggle, by Debashish Sau, named *ASL (American Sign Language) Alphabet Dataset*, has 223,000 images, about three times larger than the first one, and significantly more diverse. Before getting trained, the images also get preprocessed to 128x128 pixels, and then mirrored, angled, or recolored at random to add even further generalization to the learning in order to reduce overfitting. For hyperparameter optimization, the group analyzed batch size, epoch number, and learning rate. For batch size, each size tested hovered around ~95%-100% demonstrating size 32 and 64 as comparable highest performers for test accuracy (Figure 4). For epoch numbers, it was clear after 5-6 epochs that further epochs demonstrated diminishing returns, with test accuracies between 5-6 epochs being within ~2% of 15 epochs (Figure 5). For the learning rate, 5e-03 was clearly too slow, while 1e-03 and 1e-04 were comparable with near ~100% test accuracy (Figure 6).

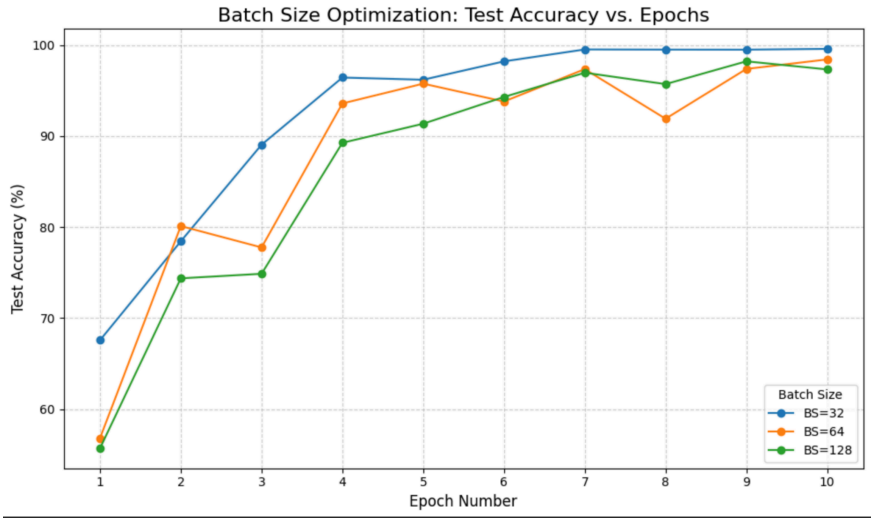


Figure 4. Test accuracy based on batch size in the convolutional neural network

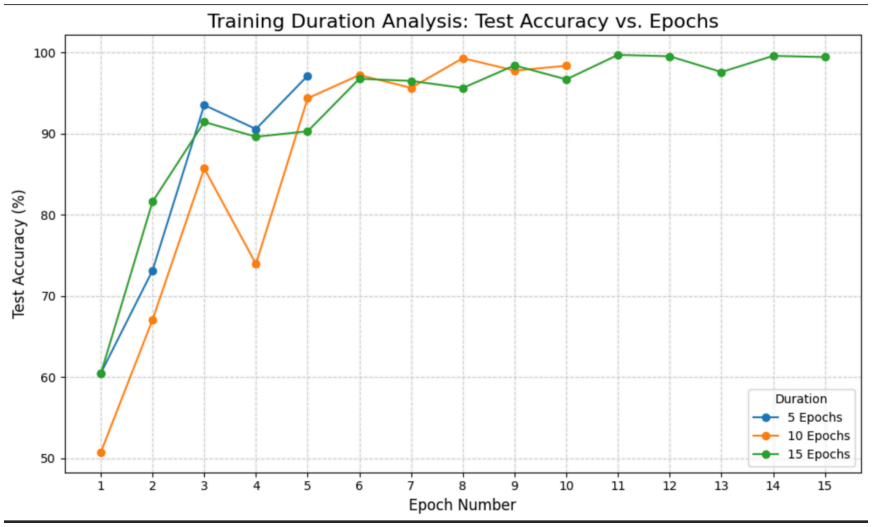


Figure 5. Test accuracy based on the number of epochs in the convolutional neural network

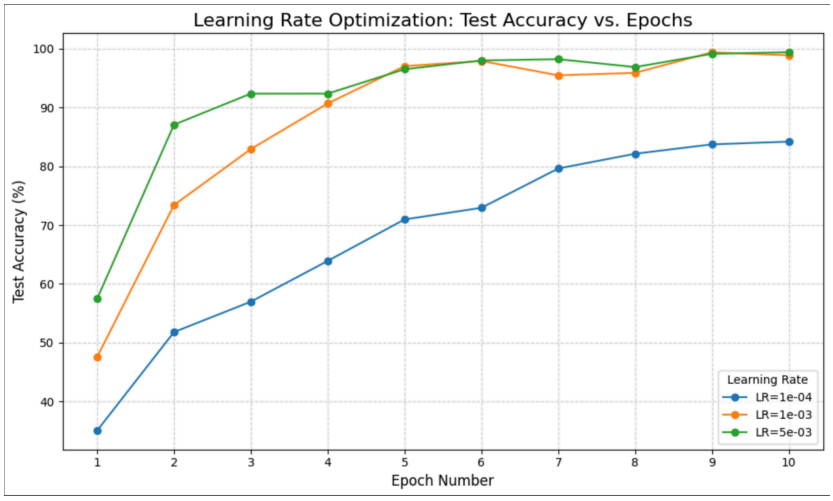


Figure 6. Test accuracy based on the learning rate in the convolutional neural network

2nd CNN

After testing multiple different model types on the initial dataset, specifically Logistic Regression, Random Forest, and a Convolutional Neural Network, the CNN gave us the highest performance out-of-sample error rate of 92.31% ± 2.87%, which is also in line with another ASL image recognition model (Garg). For hyperparameter optimization in the new CNN model, we used the same hyperparameters that showed to be most accurate in testing. These include 5-6 training epochs, which were shown to have an almost exact test accuracy compared to any greater number of epochs, while also reducing training time as the accuracy flattened (Figure 5). The chosen batch size was 32, which was shown to be more accurate than other batch sizes tested (Figure 4), and a learning rate of 1e-03, which at our chosen number of epochs (6), is almost identical to other learning rates, such as 5e-03, which both had around a 98.5% test accuracy (Figure 6).

The only implementation issue or hurdle unique to this model was the Google Colab compute limits. Due to the dataset being about three times larger than the initial one, and running a cross-validation on many epochs, each model in the hypothesis space took more than three hours to complete training, with some reaching close to seven hours, depending on the number of epochs and cross-validation folds. The Colab free tier has a compute limit per day, which was easily surpassed before even finishing training on a single model, which means the group was required to invest in a Colab Pro account to get the compute power needed to train all the CNN models on the second dataset.

To evaluate the performance of the neural network, the group compared both the out-of-sample error rates at each cross-validation fold and epoch, and classification performance using the confusion matrices with each other's model of the same structure. There is the initial

CNN trained on the first, low-variance dataset, and two CNNs trained on the second, more diverse dataset. Of the two convolutional neural networks trained on the second dataset, one used a four-fold cross-validation, and the other used six folds. The first model used four folds to be as similar as possible to the initial model and minimize differences in the comparison.

The second model used six folds in an attempt to increase the accuracy of the model, as with an increased 50% more training time due to the extra two folds alone, the model sees more images and can lower any bias. In six-fold cross-validation, the model is trained on 186,834 images per fold compared to 167,250 in four-fold cross-validation, about a 12% increase.

One of our most important qualitative features to evaluate performance is the model's ability to successfully classify our own inputted images. As said earlier, the initial dataset's reduced variance led to it not being able to classify truly out-of-sample images, such as those we can take with our computer's webcam and input right away. The new models, however, were able to classify our own inputted images at a much higher rate than the initial model.

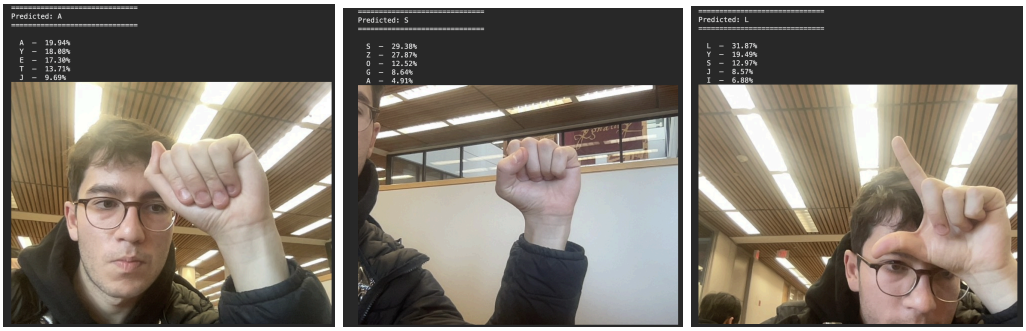


Figure 7. Three Successful Tests Spelling 'ASL' with the respective probability distribution of classes

Another qualitative metric is how often the model confuses similar signs. For example, 'M' and 'N' are the two most commonly confused letters of all 29 classes due to how similar they appear. They have roughly the same silhouette, with the same number of visible fingernails and fingers, and no visible thumb. This made us think that the features the model uses were similar to what a human brain would attribute to each hand signal, like in the example specified before. If similar hand signs are getting confused, the model has a good idea what the signal being shown is. Another example is the 'C' and 'O' signs, which have the same sign, besides the fact that the thumb and index finger are touching, and the model very often confuses a 'C' for an 'O' when inputting our own images. So it's feasible that the number of fingers up, the outline shape of the hand, or the angle the thumb and index finger make can all contribute to the final classification.

This made us curious about the actual features the model was using to classify the image, as in an image recognition model, there aren't concrete features to input, only the image, which in this case is an 128x128 pixel RGB image, so it's difficult to figure them out with 100%

certainty; we can only truly infer. The model only sees thousands of numbers that each correlate to the color of a pixel in the actual image.

To combat this, we found a library called Grad-CAM, which uses the saved model parameters to highlight the regions of the inputted image that was weighed and looked at the most to make the final prediction. Grad-CAM first runs a forward and backwards pass, similar to error back-propagation. The forward pass feeds in the image through the model to obtain the final prediction and the feature maps from the final convolutional layer. The backwards pass computes the gradient of each target class score, with respect to each feature map. This gives the "importance" as a weight for each map, indicating how much it contributes to the final class prediction (Selvaraju). With this information, specifically the feature maps and the gradients (weights), it multiplies each feature map by its corresponding weights and then sums up the values of each feature map. ReLU is used to focus on only the positive attributes, then the summed weighted feature map is resized using interpolation to the original 128x128 aspect ratio to visualize the most weighted regions used to influence the classification using a heatmap.

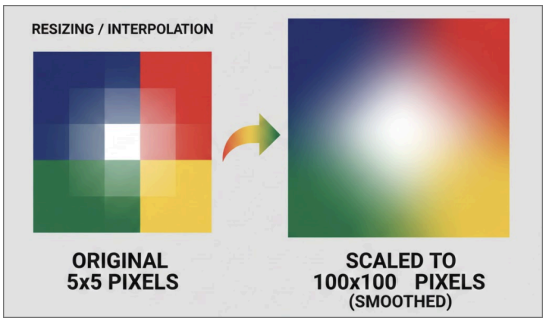


Figure 8. Graphic showing a 5x5 image using interpolation to resize into a 100x100 image

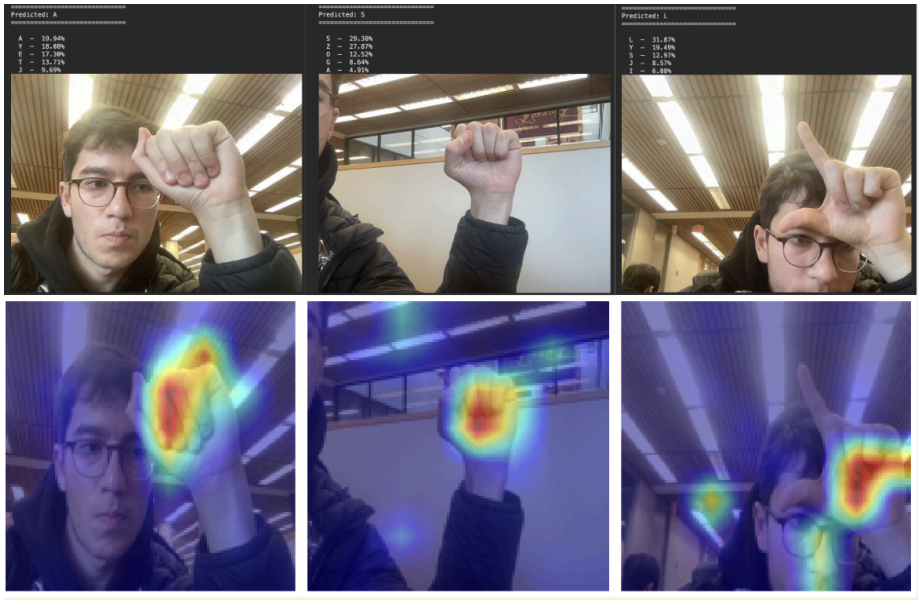


Figure 9. Grad-CAM heatmaps of signs spelling ‘ASL’. The most weighted areas in each image are the fingers and palm.

The most “looked at” regions of the images tend to always be the hands, as the model has learned to pick up on those features. We still cannot decipher with full confidence the specific aspects of the hand the model uses, but we now know it does look at the hand to influence the prediction.

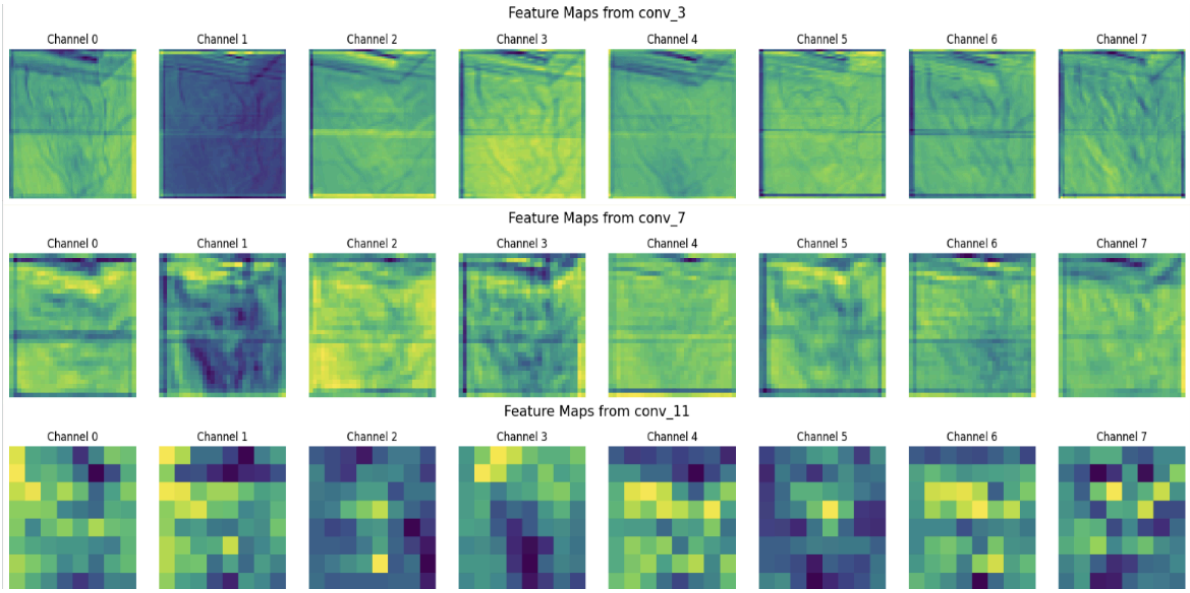


Figure 10. Visualization of layers 3, 7, 11 of the CNN model. They are the feature maps produced in the convolution layers.

Another visualization technique we utilized was the feature map visualizations, which gave us insight into the model's inner layers. They represent the internal activation patterns produced within the convolutional neural network. Every convolutional layer contains numerous learned filters, each of which becomes selectively responsive to particular visual structures. They range from vertical and horizontal lines and simple edges in early layers to more complex curves, hand-like patterns in deeper layers. When these channels are visualized as grayscale images, the varying brightness corresponds to the intensity of the filter's response across the spatial dimensions of the feature map. This is done via maxpooling, which is also why the resolution of the feature maps in each layer decreases (Zeiler and Zhang). Unlike methods such as Grad-CAM that project information back onto the original image, feature-map visualization focuses on the intermediate computations that underpin the model's decision-making process.

Results

Logistic Regression

Logistic regression achieved a mean validation accuracy of approximately 83.1% across cross-validation folds, with a standard deviation indicating consistent performance. On the final test set (20% holdout), it reached 83.1% accuracy, alongside macro-averaged precision of 83.0%, recall of 83.1%, and F1-score of 0.831, demonstrating a strong non-deep learning baseline using only flattened RGB pixels. Training accuracy was slightly higher, suggesting mild overfitting, but the model converged efficiently with mean training and validation times under a few seconds per fold. The confusion matrix revealed balanced performance across the 29 ASL classes, with minor misclassifications likely due to visual similarities in hand gestures (Figure 11).

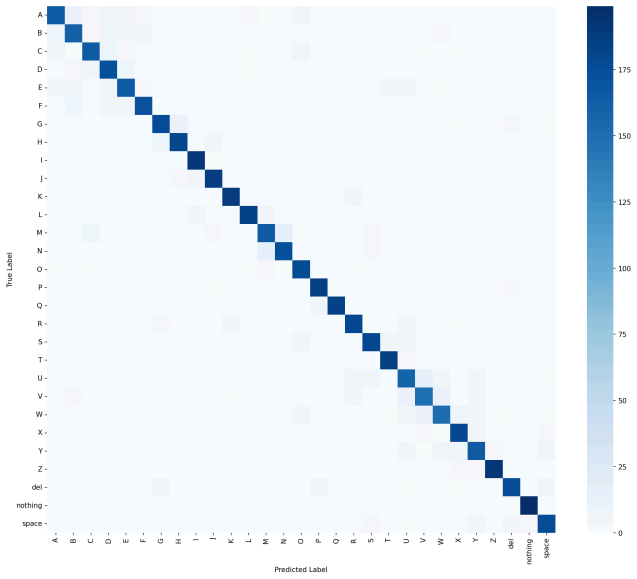


Figure 11. Confusion matrix for ASL Logistic Regression model

Random Forest

Random Forest outperformed the logistic regression baseline, attaining a mean validation accuracy of about 89.7% in cross-validation, with low variance across folds. The final model on the test set yielded 89.7% accuracy, macro-averaged precision of 89.7%, recall of 89.7%, and F1-score of 0.897, marking a notable improvement of approximately 6.6% over logistic regression. Training accuracy approached 100% due to the ensemble's capacity, while validation times remained reasonable (mean training time around seconds to minutes per fold, scaling with tree count). The confusion matrix showed enhanced class discrimination, with fewer errors in similar gestures, and the out-of-bag score supported the test results at around 89% (Figure 12). The generalization learning curve showed near-perfect training accuracy stabilizing early, with validation accuracy rising from ~10% at small dataset sizes to converging at ~90% around 65,000 samples, indicating good generalization and reduced overfitting as data volume increased (Figure 13).

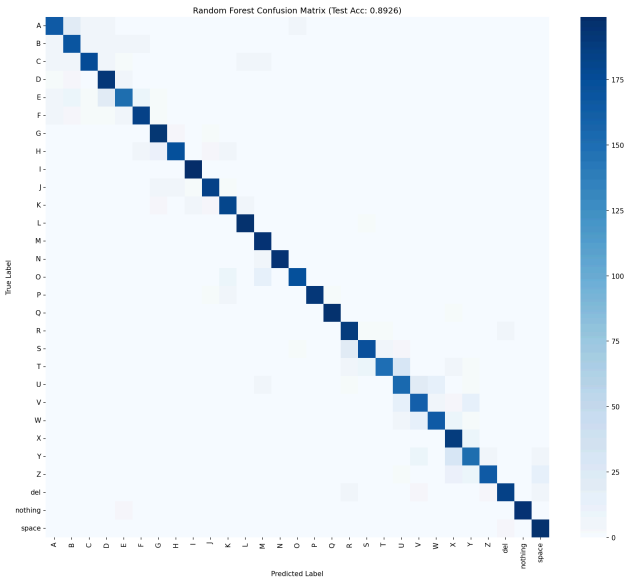


Figure 12. Confusion Matrix for ASL Random Forest model

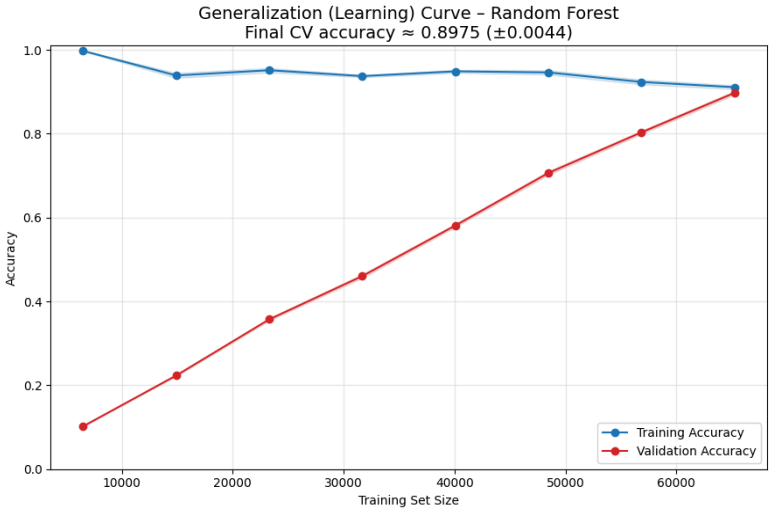


Figure 13. Generalization Learning Curve for the ASL Random Forest model

1st CNN Model

We used 4-fold stratified cross-validation (Figure 14) to obtain an estimate of the model's out-of-sample performance. Across the four folds, the model achieved a mean accuracy of 92.31% ± 2.87% and a mean loss of 0.2839 ± 0.0100. A random predictor would be expected to have an accuracy of 1/29 or 3.45% and a cross-entropy loss of 3.37.

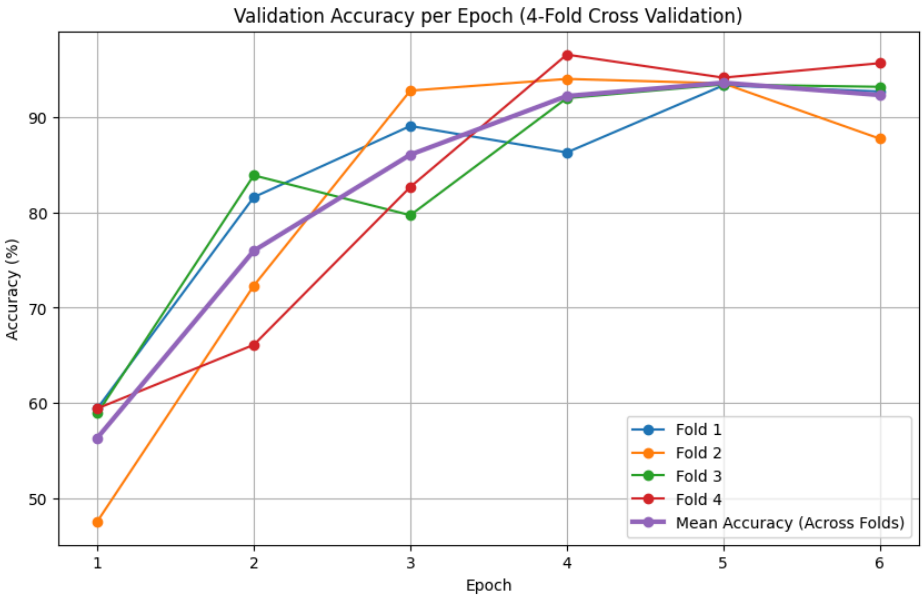


Figure 14. 4-fold cross-validated accuracy per epoch

The generalization curve (Figure 15) shows that our out-of-sample error rate is lowest at the 5th epoch, indicating a slight amount of overfitting. The blue line represents the OOS error, which is calculated as 1 minus the accuracy, scaled between 0 and 1. The in-sample error rate is calculated as cross-entropy loss. This is why the in-sample error rate is above the out-of-sample; they are representing two distinct things. In methods, we discussed how we chose 6 epochs, and although this result shows signs of overfitting, we are confident that as we increase the size of the dataset, the model will be forced to generalize more.

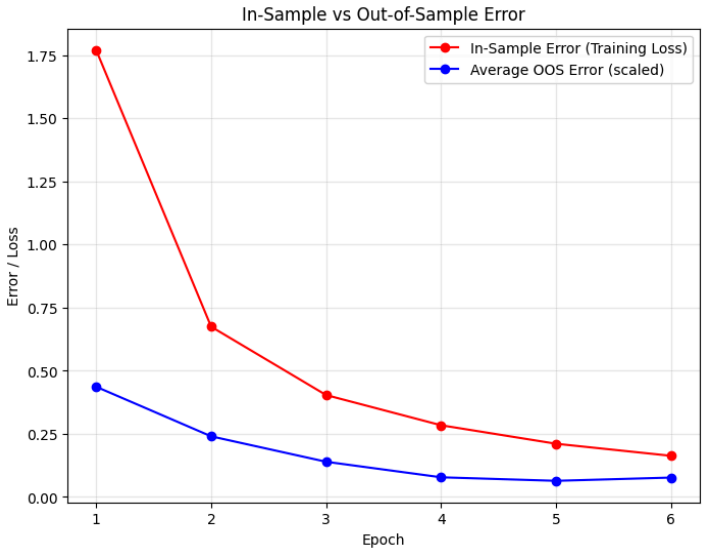


Figure 15. Generalization curve over epochs

The confusion matrix (Figure 16) provides more detailed insight per class. We see that most classes are very accurate, with some classes like A, M, and Z being confused regularly. This is likely due to the similarities they share with other signs and the fact that Z is a moving hand sign. We suspect that with a new dataset with a more diverse sample size, the model will be more accurate in these classes.

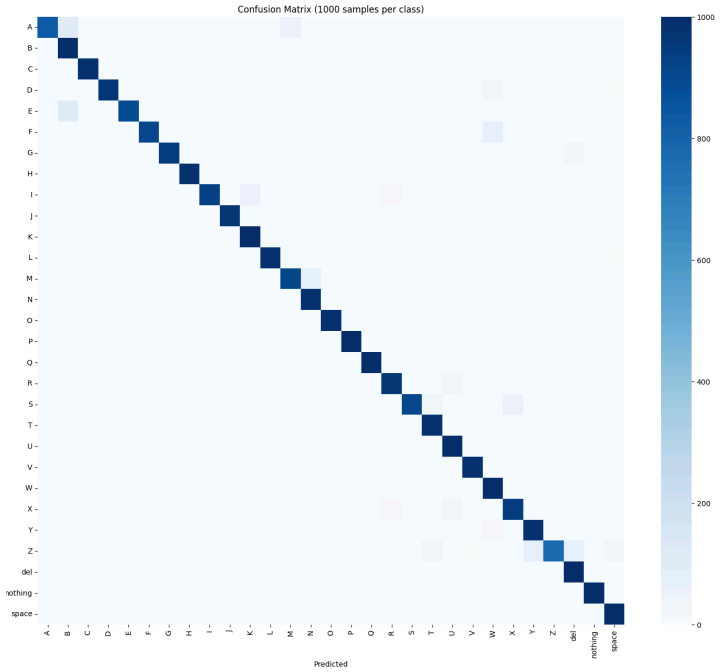


Figure 16. Confusion matrix of all 29 classes

Although we met our goal of surpassing 90% accuracy, the model was not highly accurate when identifying our own hands. After testing, we found that the model only predicted correctly when we intentionally mimicked the exact positioning and style of the person in the dataset. This indicates that the dataset, and therefore the model, does not generalize well to new users unless their signs closely resemble the dataset's examples. Due to this limitation, and after observing the architecture's performance during training, we proceeded by applying the same CNN structure to a new, more diverse dataset.

2nd CNN Model

In general, comparing each CNN by their respective amount of training, the models that were trained on more data performed better, using the evaluation metrics of cross-validated out-of-sample error rates and confusion matrices. The first CNN trained on the second dataset had a final mean accuracy of 92.88% ± 0.54%, and the second had 97.39% ± 0.53%.

Comparing just this first model's accuracy of 92.88% ± 0.54% to the initial model's accuracy of 92.31% ± 2.87%, it doesn't appear to be very different at all. This is because both models have the same structure and hyperparameters, only trained on different datasets, so they generalize similarly given their respective datasets. This shows that the error rates from training and validating on the same dataset don't tell the whole story, as, despite having very similar error rates, the second model performs remarkably better in classifying truly out-of-sample images, such as the ones our group took and inputted ourselves. Using our own images, the initial model seemed to be no better than a random guessing model, while the model trained on the second dataset tended to successfully classify the signs, or at least classify them as a commonly confused class a majority of the time, noting that the images used had clear lighting and legible signs, which shows that the second dataset's richness contributes to the decrease of out-of-sample error.

Now comparing the two models that were trained on the diverse second dataset, the model trained on more data performed better using cross-validated error rates, by about 5%. It trained on about 12% more data per fold, and trained for 50% more total epochs (from four to six) in order to increase the accuracy by this much. This explains that despite the richness or complexity of the dataset, the amount of training directly correlates to the success of the model.

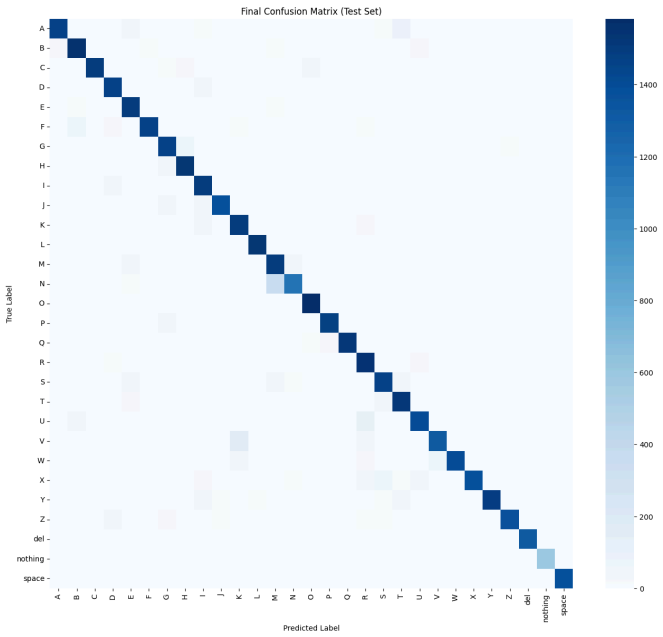


Figure 17. Confusion Matrix for first CNN model (4 epochs)

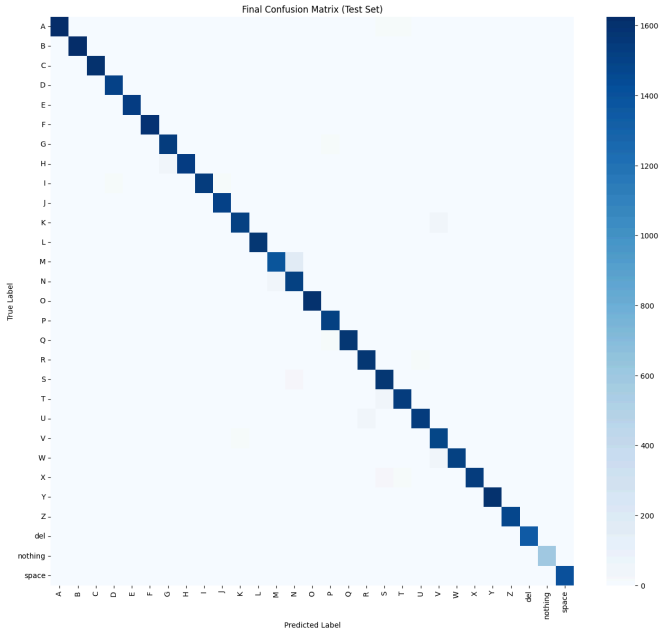


Figure 18. Confusion Matrix for the second CNN model (6 epochs)

These confusion matrices show the difference in how much each model confuses letters with the correct label. Figure 17 shows a common confusion between ‘M’ and ‘N’, and a few outlier confusions, such as with ‘K’ and ‘V’, which have a very similar hand sign, while Figure 18 clearly shows a decrease in these misclassifications, as the matrix is more diagonal than the first one. This is due to the increased amount of training the model in figure 18 went through than the model in figure 17, which lets it generalize more accurately.

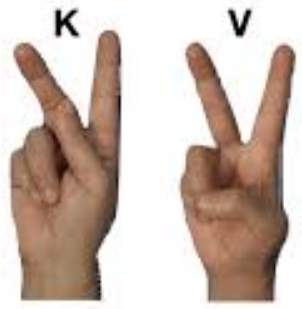
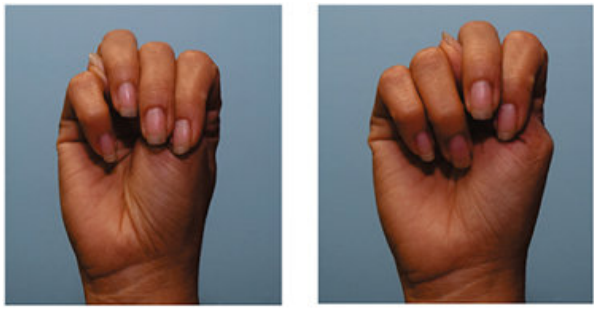


Figure 19. ASL hand signs of ‘K’ and ‘V’



(a) Letter M (b) Letter N

Figure 20. ASL hand signs of ‘M’ and ‘N’

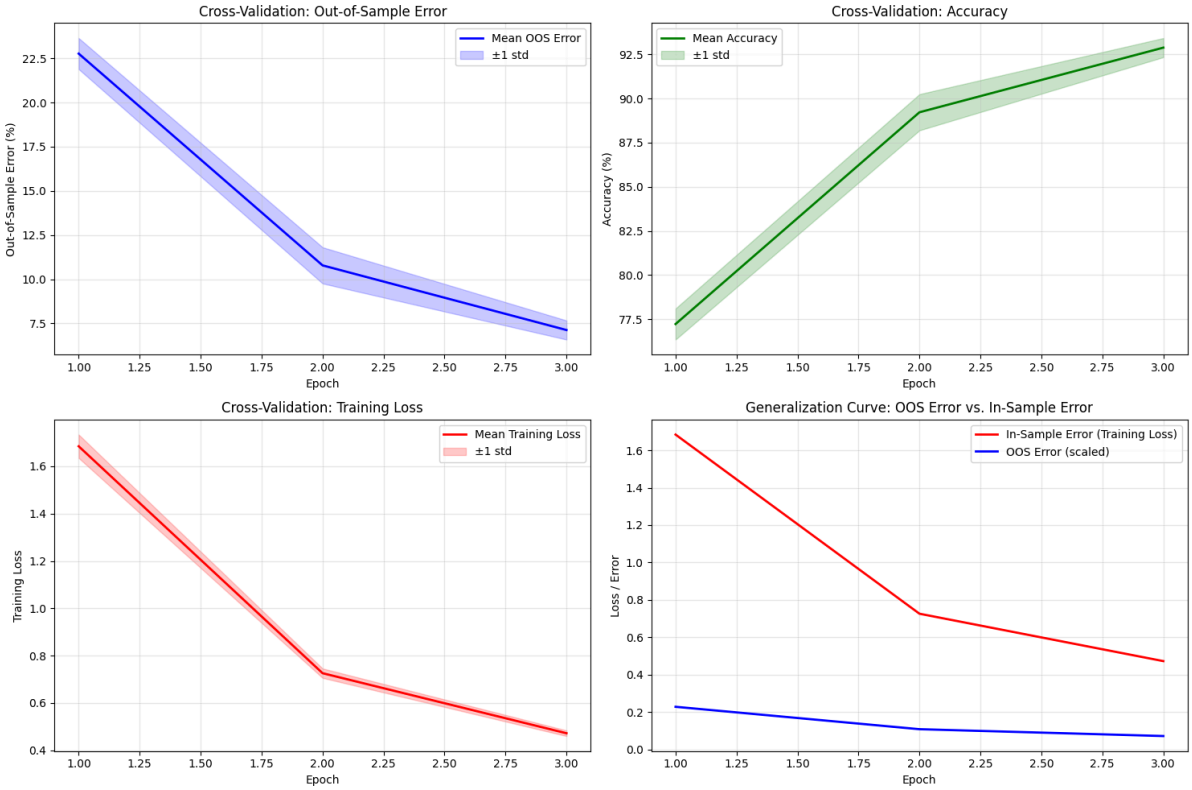


Figure 21. 4-fold cross-validated error for the first CNN model (4 epochs)

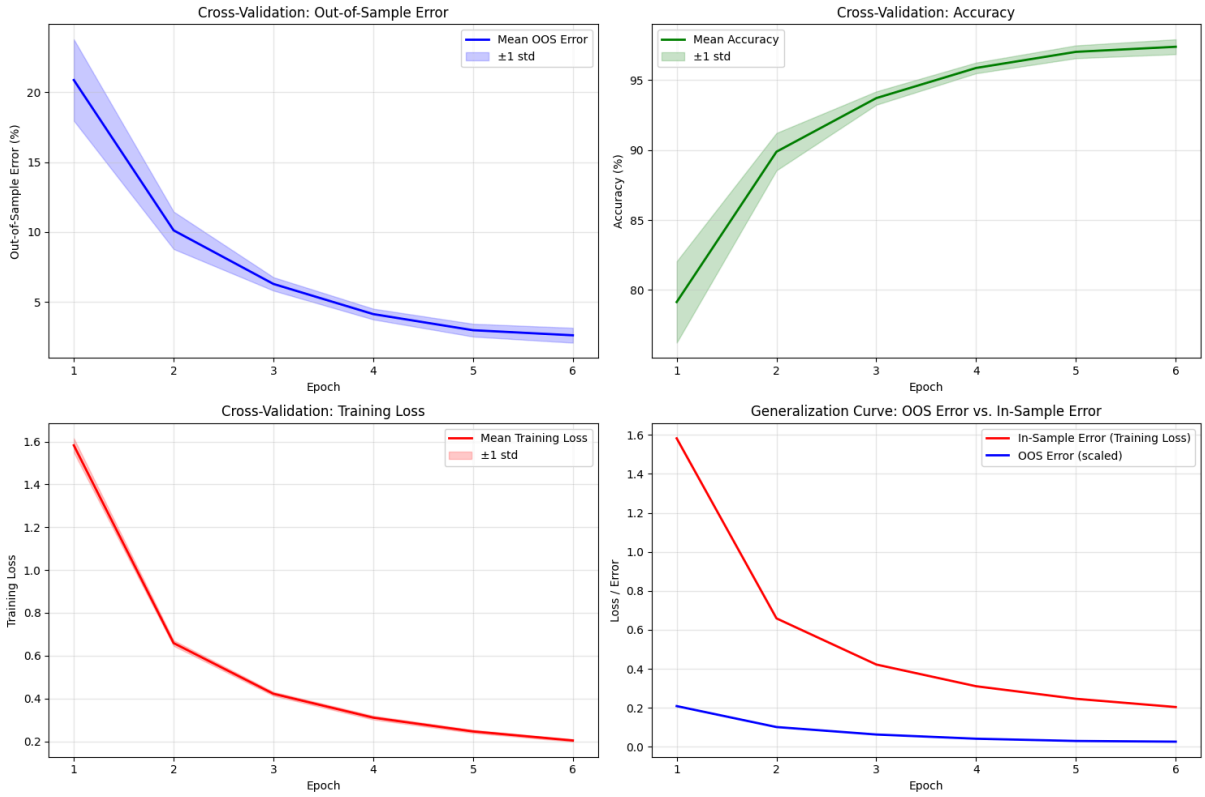


Figure 22. 6-fold cross-validated error for the second CNN model (6 epochs)

These graphs portray the effect of additional training on the accuracy of the model. If you compare the accuracy of each model at each epoch, the metrics are very similar. However, due to the second model training under twice the number of epochs, it was able to increase its accuracy further than three epochs ever could. The error rates also appeared to start to flatten out around an asymptote in the sixth epoch, showing that any extra training could potentially cause the model to overfit the data, which makes sense, looking back at Figure 22, where an increased number of training epochs could potentially detrimentally affect the model's performance.

Conclusion

The main take-home message from this project is that each person has their own way of signing as well as skin color, handedness, background, and each photo will be very different. For any image classification model that is trained, especially an ASL one that is attempting to generalize for any person, the number of unique samples in the dataset is crucial to prevent overfitting in the model to one person or a group of people.Our group achieved an accuracy of roughly 97.39% ± 0.53% with our final CNN model, which surpassed our original goal of 90% classification accuracy. The most important factor that contributed to the success of the model was the richness and variance of the second dataset. Comparing the in-sample rates of both, the

models trained on either dataset showed similar metrics given similar hyperparameters; however, the out-of-sample metrics illustrated the importance of having variability in the training data for the model to generalize from. If we attribute the “success” of the model to its out-of-sample error, which is what’s important in the real world, then the second dataset gives night and day results when testing real images outside the dataset. In the initial dataset trained on a single person’s hand in the same room, if you didn’t mimic those images, the model was no better than a random guess at classifying the image, but using the second rich dataset, the model can classify almost any image correctly, given it’s legible.

In retrospect, with our current experience, we would have transitioned to the more complex, real-world dataset much earlier in the process. Our initial time spent training baseline models (Logistic Regression, Random Forest) and our first CNN on the simpler dataset, while educationally valuable, delayed our time with the core challenges of realistic data variance and background noise that the final model had to overcome.There are two improvements that we can make for future work, the first of which is to incorporate Python's MediaPipe package since it tracks finger orientation and overall hand position, which can help the model identify what symbol the hand is making. The second change is that ASL is very fast and includes moving parts. Ideally, you would want an ASL classifier to be able to take in multiple photos in a row and identify movement. The letters ‘J’ and ‘Z’ include movement that can’t be captured in a static image. If we wanted to improve on this work, add more vocabulary to the model, we would want to find a way to use MP4s or GIFs instead of JPGs for the model.

In closing, this project successfully showed that a well-designed, tested, and implemented Convolutional Neural Network, when trained on highly diverse data, can output a highly accurate prediction for classifying images of American Sign Language hand signs, confirming that our final CNN model provides a generalized and reliable solution capable of performing outside of controlled environments.

Bibliography

Garg, Bindu; Kasar, Manisha; Paygude, Priyanka; Dhumane, Amol; Ambala, Srinivas; Rajpurohit, Jitendra; Sharma, Abhay; Meshram, Vidula; Vats, Amber; Kashyap, Achyut. “Sign Language Detection Dataset: A Resource for AI-Based Recognition Systems.” Data in Brief, Volume 61, August 2025, Article 111703.

Selvaraju, Ramprasaath R., Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh & Dhruv Batra (2016). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.

B. Alsharif, E. Alalwany, A. Ibrahim, I. Mahgoub, and M. Ilyas, "Real-Time American Sign Language Interpretation Using Deep Learning and Keypoint Tracking," *Sensors (Basel)*, vol. 25, no. 7, 2138, Mar. 2025. doi: 10.3390/s25072138.

C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

L. Breiman, "Random Forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

M. J. Cheok, Z. Omar, and M. H. Jaward, "A review of hand gesture and sign language recognition techniques," *Int. J. Mach. Learn. Cybern.*, vol. 10, no. 1–3, pp. 249–262, Jan. 2019. doi: 10.1007/s13042-017-0705-5.

O. Koller, S. Zargaran, H. Ney, and R. Bowden, "Deep Sign: Enabling Robust Statistical Continuous Sign Language Recognition via Hybrid CNN–HMM Models," *Eng. Appl. Artif. Intell.*, vol. 72, pp. 225–241, 2018.